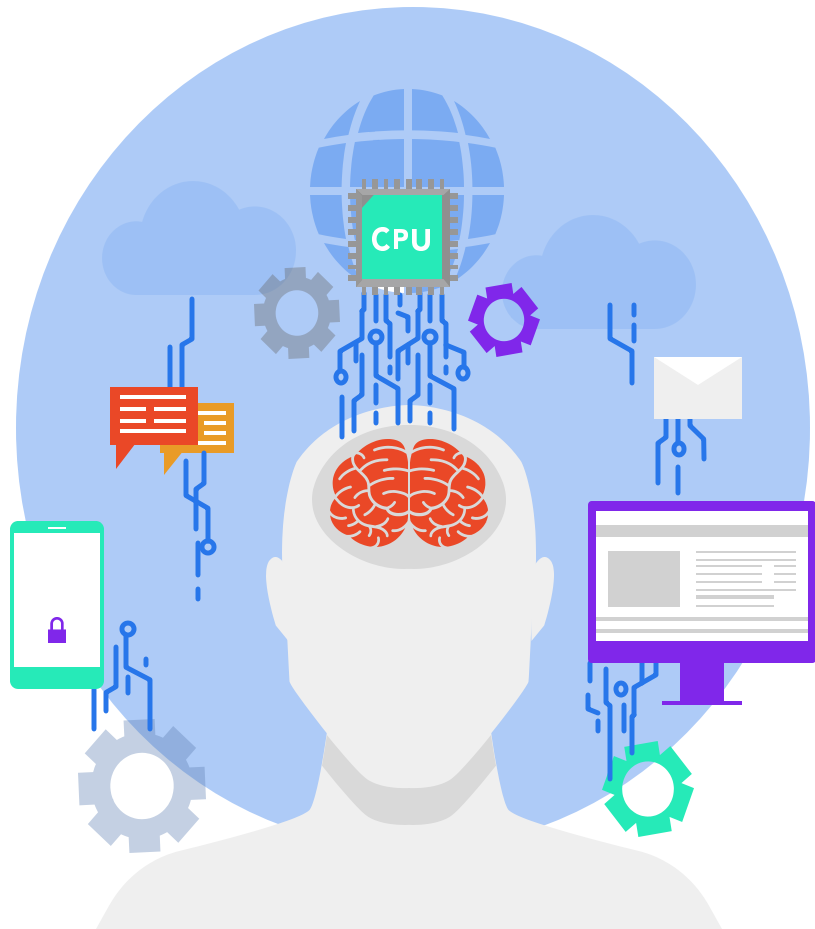




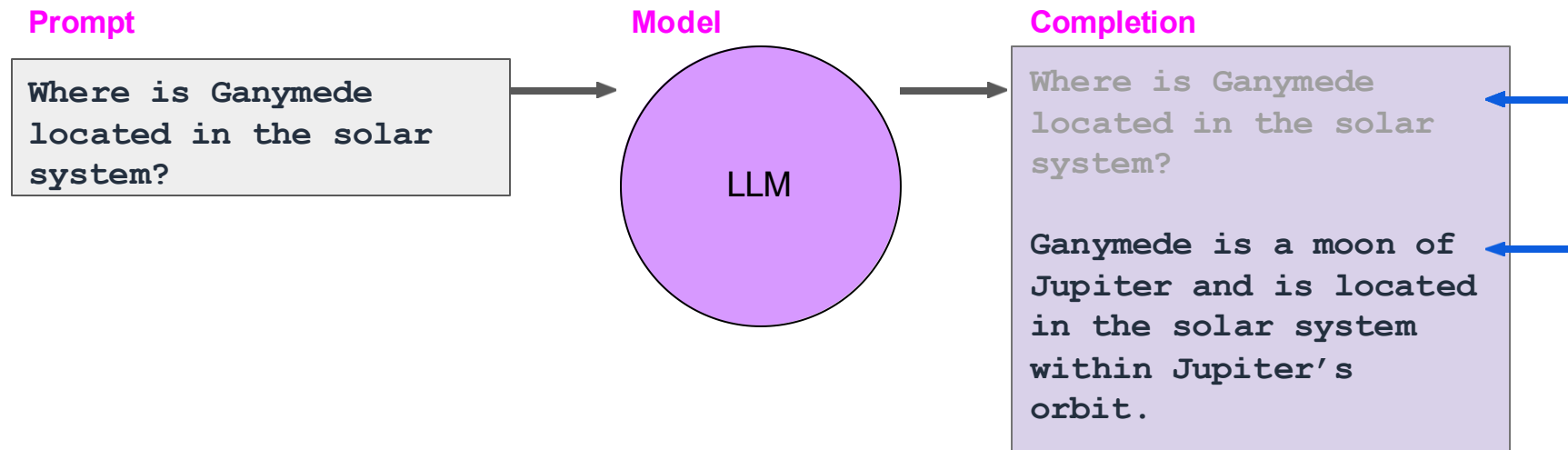
Introduction to Generative AI and LLMs

Dr. Tamer Arafa

What is generative AI

- Generative AI represents a major shift in artificial intelligence—from analysis to creation. These systems don't just interpret information; they generate new possibilities.
- By training on massive datasets, they learn underlying structures of language, imagery, and sound, allowing them to produce highly realistic and useful content.
- This capability extends across industries—from personalized marketing and design to scientific discovery—fundamentally changing how we create and innovate.

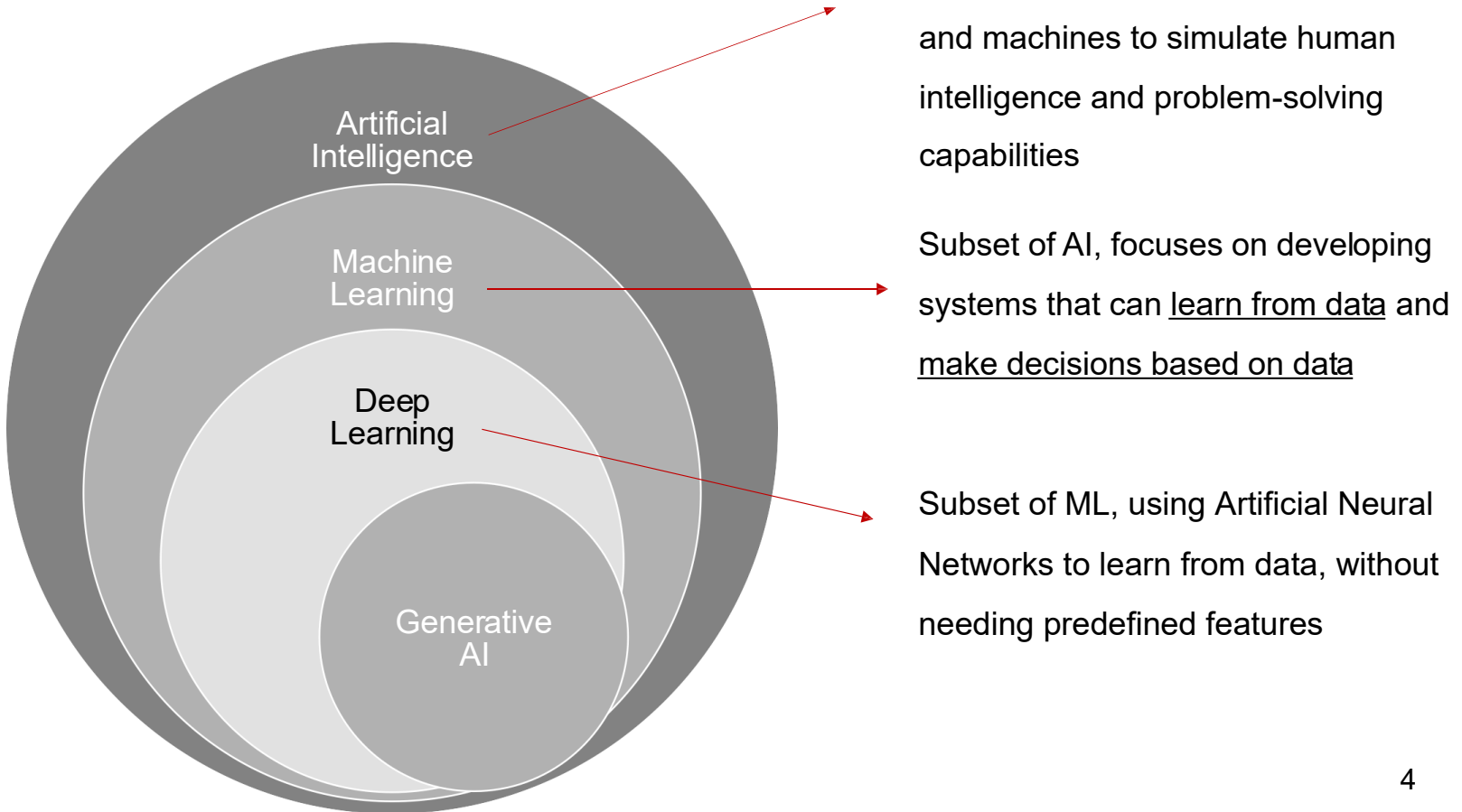
Prompts and completions



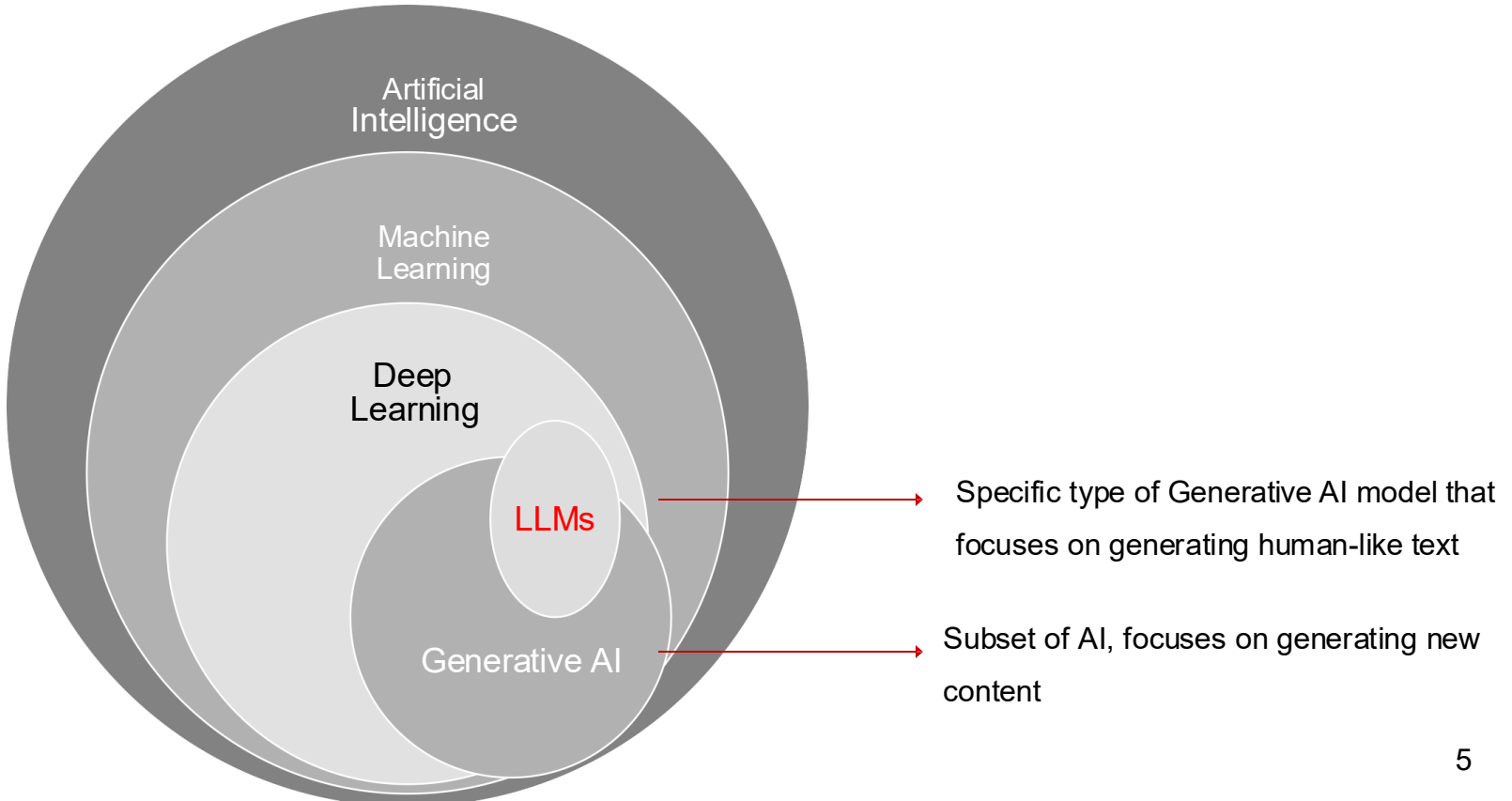
Context window

- typically a few 1000 words.

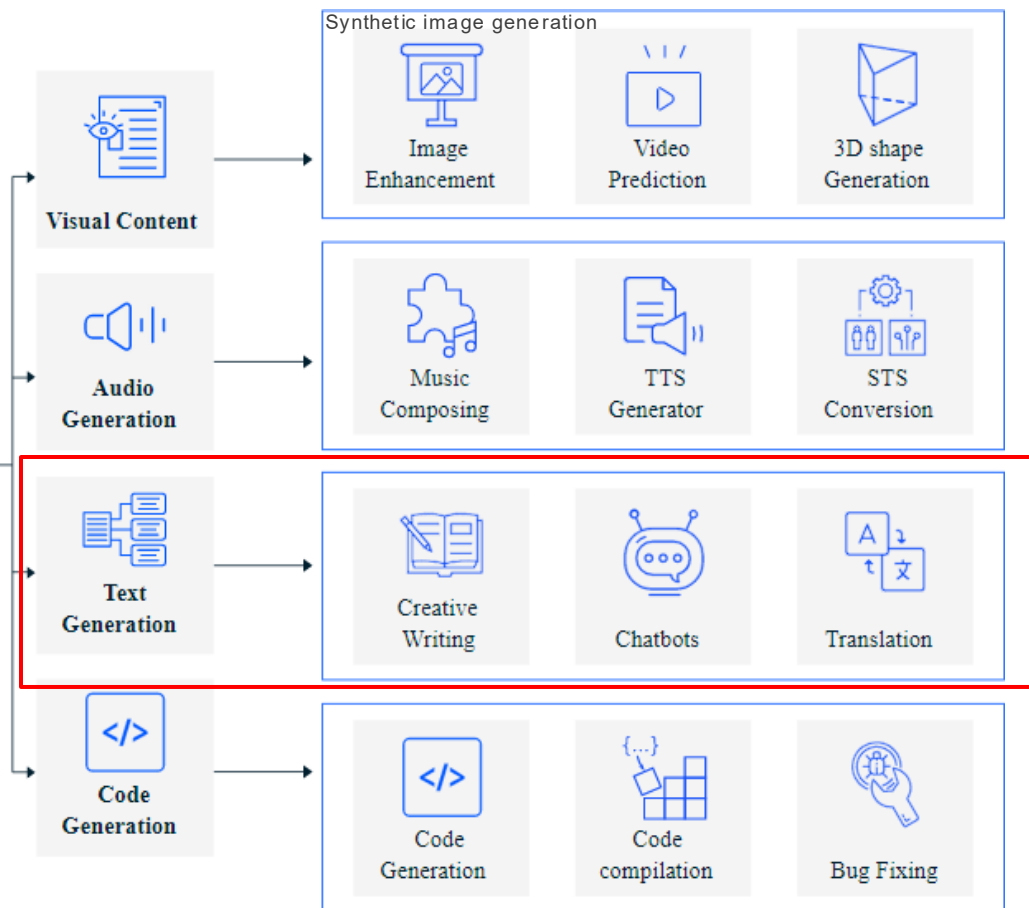
AI Taxonomy



AI Taxonomy



**Generative
AI use cases**

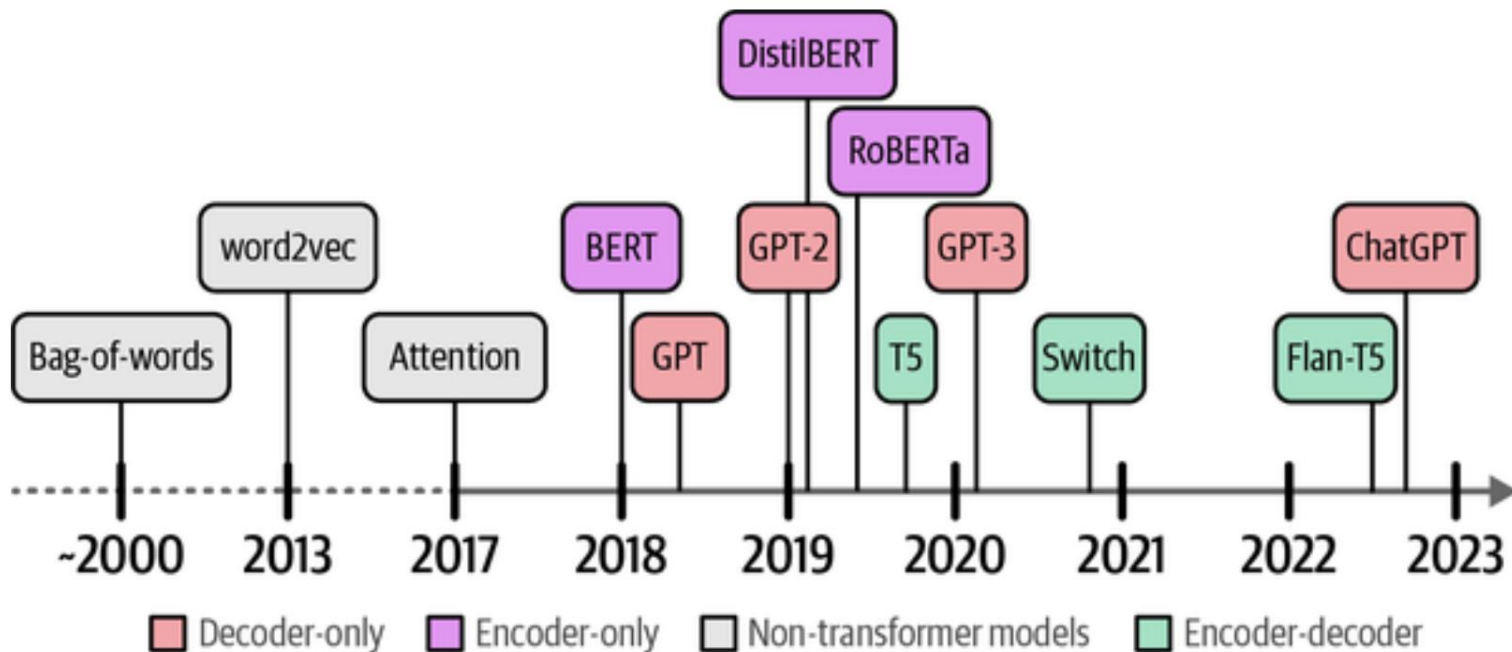


Language AI & Language Models

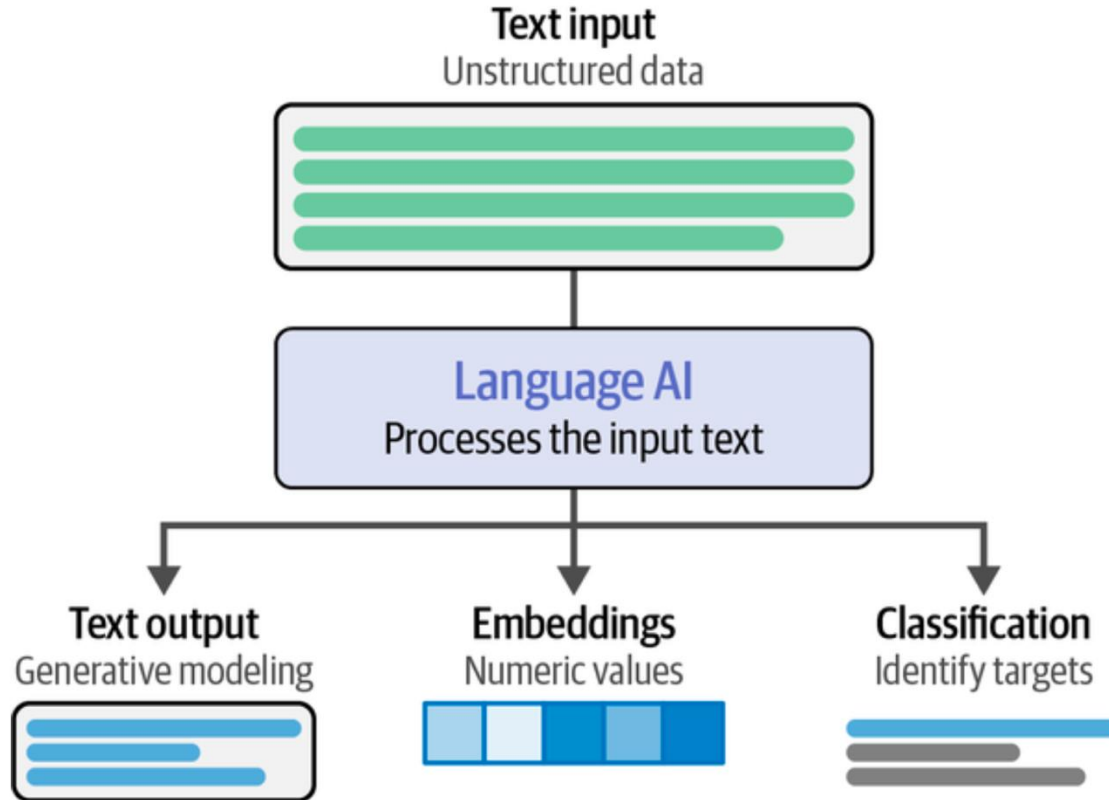
What Is Language AI?

- Language AI refers to a subfield of AI that focuses on developing technologies capable of understanding, processing, and generating human language.
- The term Language AI can often be used interchangeably with natural language processing (NLP)

The History of Language AI



Language AI Capabilities



The language modeling problem

A language model answers the question: **What is $p(\text{text})$?**

Text is a sequence of symbols: $(x^{(1)}, x^{(2)}, \dots, x^{(N)})$

$$p(x^{(1)}, x^{(2)}, \dots, x^{(N)})$$

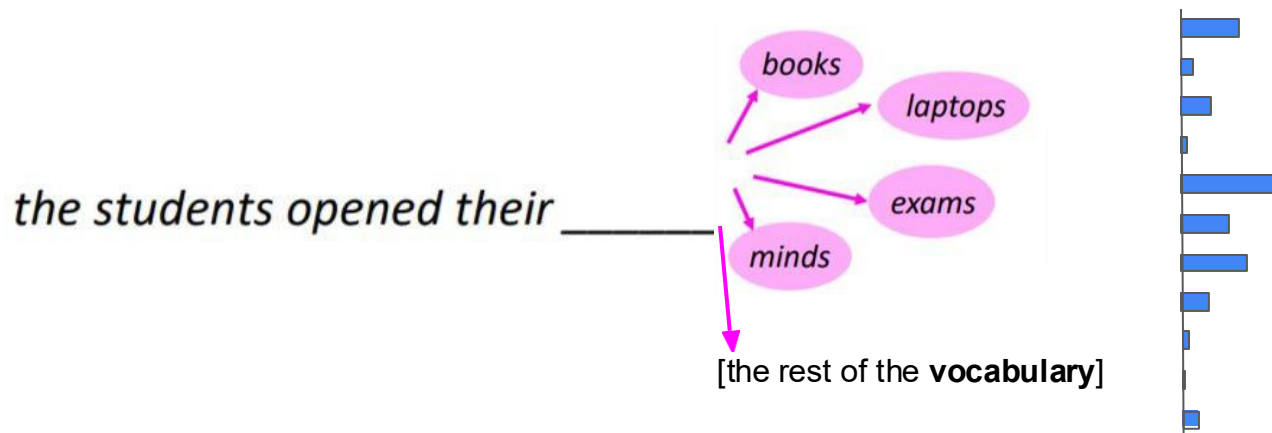
$$p(x^{(1)})p(x^{(2)}|x^{(1)})p(x^{(3)}|x^{(1)}, x^{(2)}) \dots$$

$$\prod_{i=1}^N p(x^{(i)} | \underbrace{x^{(1)}, \dots, x^{(i-1)}}_{\text{context}})$$

Just the chain rule of probability— no simplifying assumptions!

The language modeling problem

$$\prod_{i=1}^N p(x^{(i)} | \underbrace{x^{(1)}, \dots, x^{(i-1)}}_{\text{context}})$$



How do Language Models Work

- Next-word prediction: Language models estimate the probability of the next word given previous words in a sequence.
- Markov assumption: They simplify this by assuming each word depends only on a limited number of previous words (e.g., the last one or few), not the entire history.
- Learning from data: The model learns these conditional probabilities from large text datasets and uses them to generate or evaluate new text sequences

Markov assumption

- We make the **Markov assumption**: $\mathbf{x}^{(t+1)}$ depends only on the preceding **n-1** words

$$P(\mathbf{x}^{(t+1)} | \mathbf{x}^{(t)}, \dots, \mathbf{x}^{(1)}) = P(\mathbf{x}^{(t+1)} | \underbrace{\mathbf{x}^{(t)}, \dots, \mathbf{x}^{(t-n+2)}}_{\text{n-1 words}})$$

assumption

Markov assumption

- $P(\text{the} \mid \text{its water is so transparent that}) \cong P(\text{the} \mid \text{transparent that})$
or maybe even
- $P(\text{the} \mid \text{its water is so transparent that}) \cong P(\text{the} \mid \text{that})$

n-gram Language Models

$$\prod_{i=1}^N p(x^{(i)} | x^{(1)}, \dots, x^{(i-1)})$$

- *“I have a dog whose name is Lucy. I have two cats, they like playing with Lucy.”*
- **Question:** How to teach a Language Model?
- **Answer** (pre- Deep Learning): teach an *n-gram* Language Model!
- **Idea:** Collect statistics about how frequent different n-grams are and use these to predict next word

unigram probability

- *“I have a dog whose name is Lucy. I have two cats, they like playing with Lucy.”*
- corpus size $m = 17$
- $P(\text{Lucy}) = 2/17$; $P(\text{cats}) = 1/17$
- Unigram probability:
$$P(w) = \frac{\text{count}(w)}{m} = \frac{C(w)}{m}$$

bigram probability

- *“I have a dog whose name is Lucy. I have two cats, they like playing with Lucy.”*

$$P(A | B) = \frac{P(A,B)}{P(B)}$$

$$P(\text{have} | I) = \frac{P(I \text{ have})}{P(I)} = \frac{2}{2} = 1$$

$$P(\text{two} | \text{have}) = \frac{P(\text{have two})}{P(\text{have})} = \frac{1}{2} = 0.5$$

$$P(\text{eating} | \text{have}) = \frac{P(\text{have eating})}{P(\text{have})} = \frac{0}{2} = 0$$

$$P(w_2|w_1) = \frac{C(w_1, w_2)}{\sum_w C(w_1, w)} = \frac{C(w_1, w_2)}{C(w_1)}$$

trigram probability

- *“I have a dog whose name is Lucy. I have two cats, they like playing with Lucy.”*

$$P(A | B) = \frac{P(A,B)}{P(B)}$$

$$P(a | \text{I have}) = \frac{C(\text{I have a})}{C(\text{I have})} = \frac{1}{2} = 0.5$$

$$P(\text{several} | \text{I have}) = \frac{C(\text{I have several})}{C(\text{I have})} = \frac{0}{2} = 0$$

$$P(w_3 | w_1 w_2) = \frac{C(w_1, w_2, w_3)}{\sum_w C(w_1, w_2, w)} = \frac{C(w_1, w_2, w_3)}{C(w_1, w_2)}$$

n-gram probability

- *“I have a dog whose name is Lucy. I have two cats, they like playing with Lucy.”*

$$P(A | B) = \frac{P(A,B)}{P(B)}$$

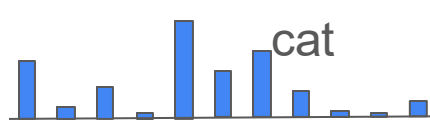
$$P(w_i | w_1, w_2, \dots, w_{i-1}) = \frac{C(w_1, w_2, \dots, w_{i-1}, w_i)}{C(w_1, w_2, \dots, w_{i-1})}$$

Large Language Models (LLMs)

- The transformer model allows fast parallel computations on many GPUs (large amounts of compute)
- It allows training on large amounts of data (think the whole internet worth of text).
- It allows adding many and many layers in the model (large model)
- A large language model is a language model with a large number of parameters, trained on large amounts of data, for long period of time.

Neural language models

$$\prod_{i=1}^N p(x^{(i)} | x^{(1)}, \dots, x^{(i-1)})$$



A distribution over
the vocabulary

$f(\Theta)$

A differentiable
function (e.g. a
neural network)

I have a dog whose name is Lucy. I have two

Neural language models

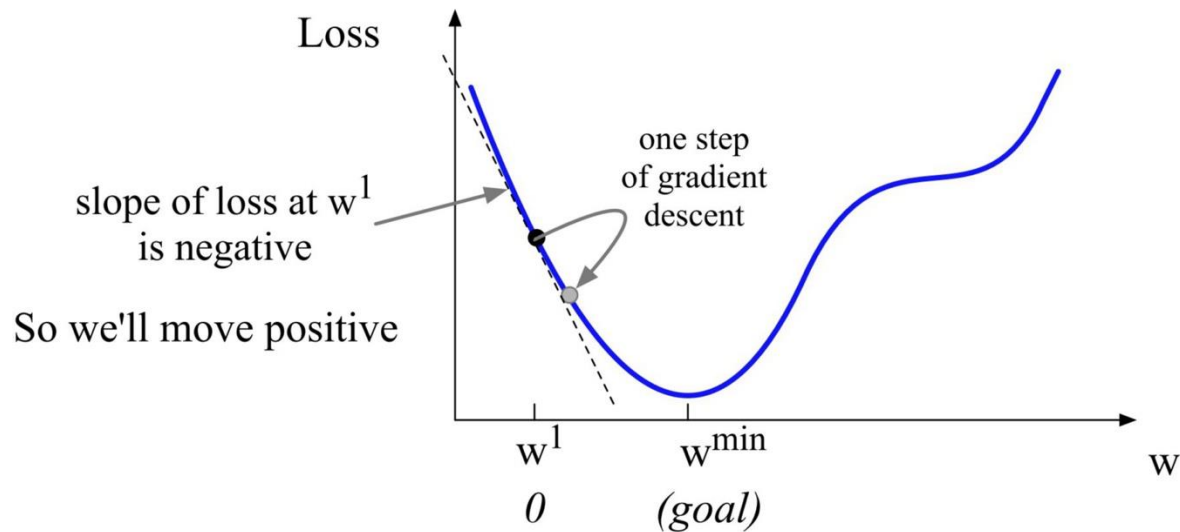
- Vocab (): {dogs, cats, bananas}
- Model logits (from $f(\theta)$) : [2.1, 1.3, -0.5]
- Apply softmax $\rightarrow$ probabilities:
 - dogs: $e^{2.1}/\sum \approx 8.17/12.44 \approx 0.66$
 - cats: $e^{1.3}/\sum \approx 3.67/12.44 \approx 0.30$
 - bananas: $e^{-0.5}/\sum \approx 0.61/12.44 \approx 0.05$
- Generation: Greedy pick $\rightarrow$ “dogs”.
- Backprop updates θ to make “dogs” even more likely next time in similar contexts.

How do we maximize the likelihood?

- The dominant strategy from the past decade:
 1. The randomly initialized differentiable function (neural network) takes the context as **input**
 2. Have that function output a probability distribution over the vocabulary
 3. Treat the **probability** of the correct token as your objective to maximize.
 4. Or **negative log (probability)** as your objective to **minimize**
 5. Differentiate with respect to the parameters, and perform **gradient descent**

Gradient Descent

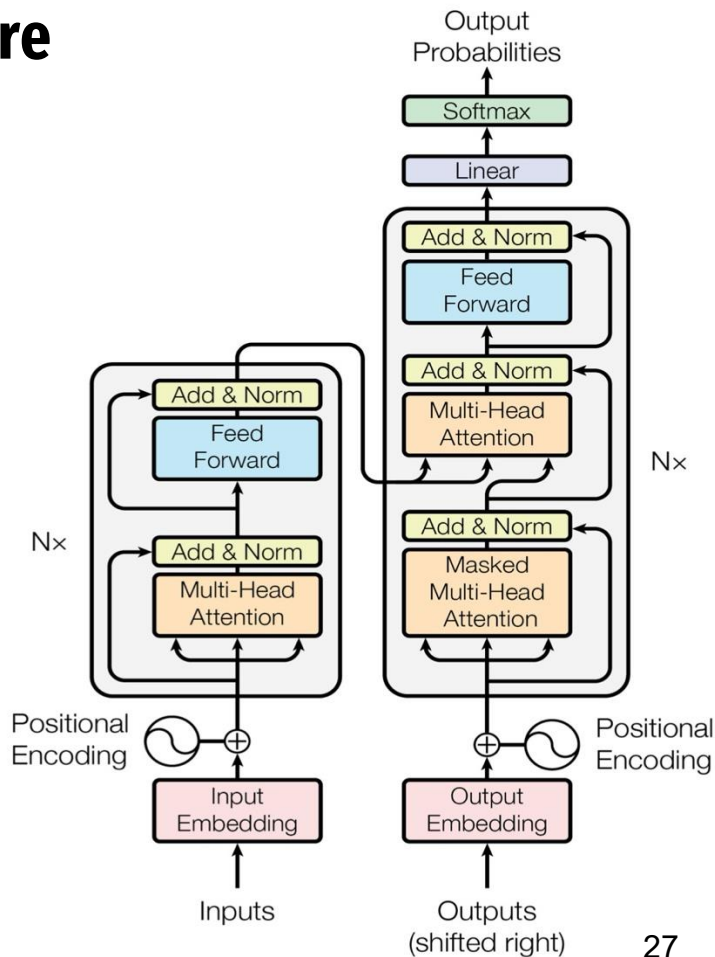
- Q: Given current parameter w , should we make w bigger or smaller to minimize our loss?
- A: Move w in the reverse direction from the slope of the function



Large Language Models

LLM Architecture

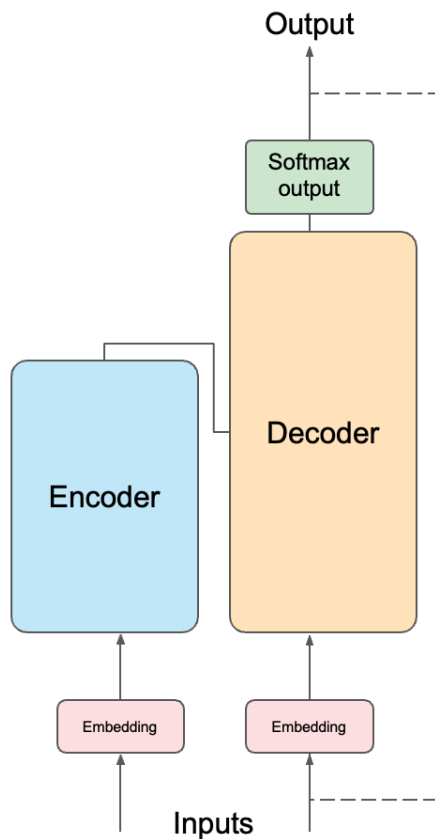
- LLM is a type of transformer model
 - Transformer:
- A neural network that learns context and meaning by tracking relationships in sequential data



Transformer Architecture

Encoder

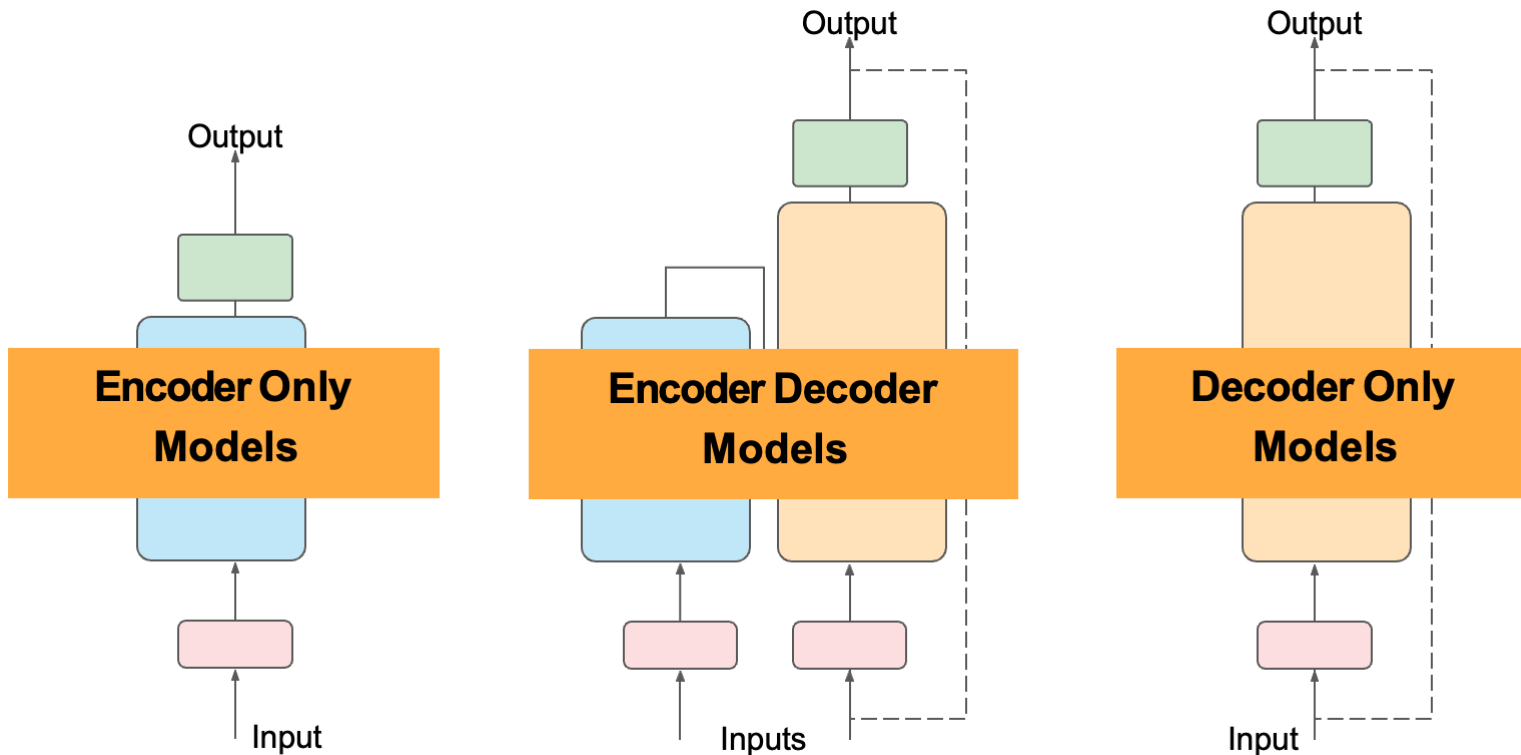
Encodes inputs (“prompts”) with contextual understanding and produces one vector per input token.



Decoder

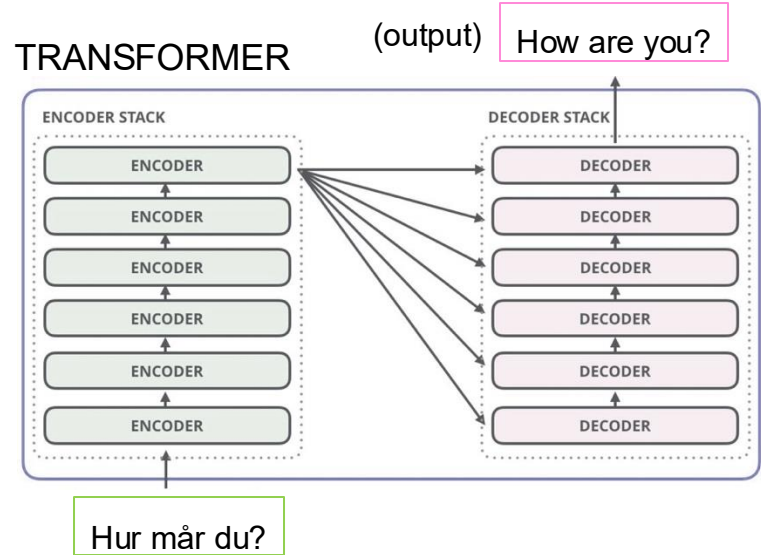
Accepts input tokens and generates new tokens.

Transformer Architecture



Transformer Architecture

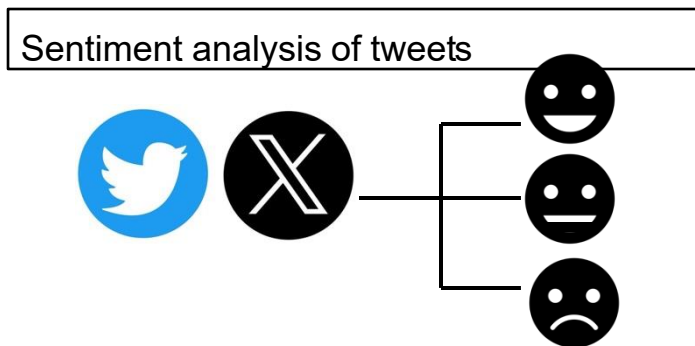
- Encoder-decoder architecture
- Encoder extracts features from an input sequence
- Decoder uses the features to produce an output sentence



Case: translation

Encoder Only LLMs

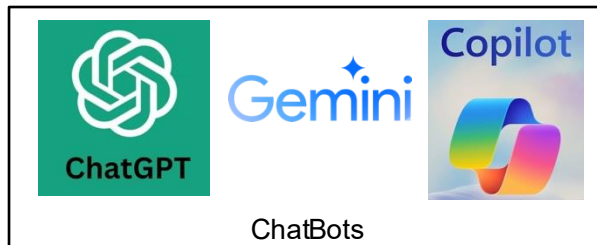
- Suited for tasks that can understand language,
- such as toxicity classification and sentiment analysis.



Example: BERT (Bidirectional Encoder Representations from Transformers)

Decoder Only LLMs

- Suited for generating language and content,
- such as story writing, blog generation, open-domain Q/A and virtual assistants.



Example: GPT-3 (Generative Pretrained Transformer 3)

Encoder Decoder LLMs

- Combine the encoder and decoder components of the transformer architecture
- Suited for both understanding and generating content,
- such as translation, text-to-code and summarisations.
- Example: T5 (Text-to-Text Transformers) by HuggingFace



Translation



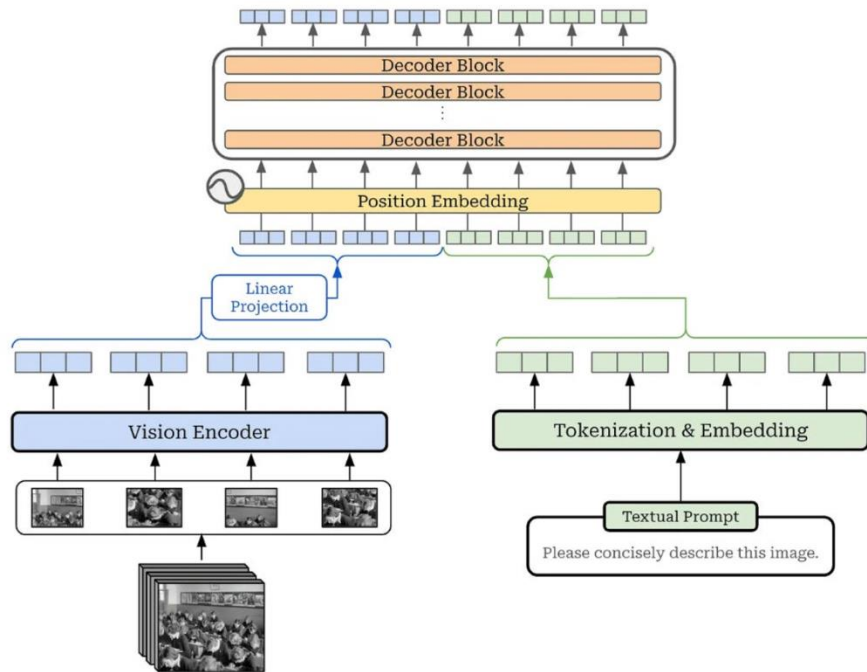
Summarization

Multimodal LLMs

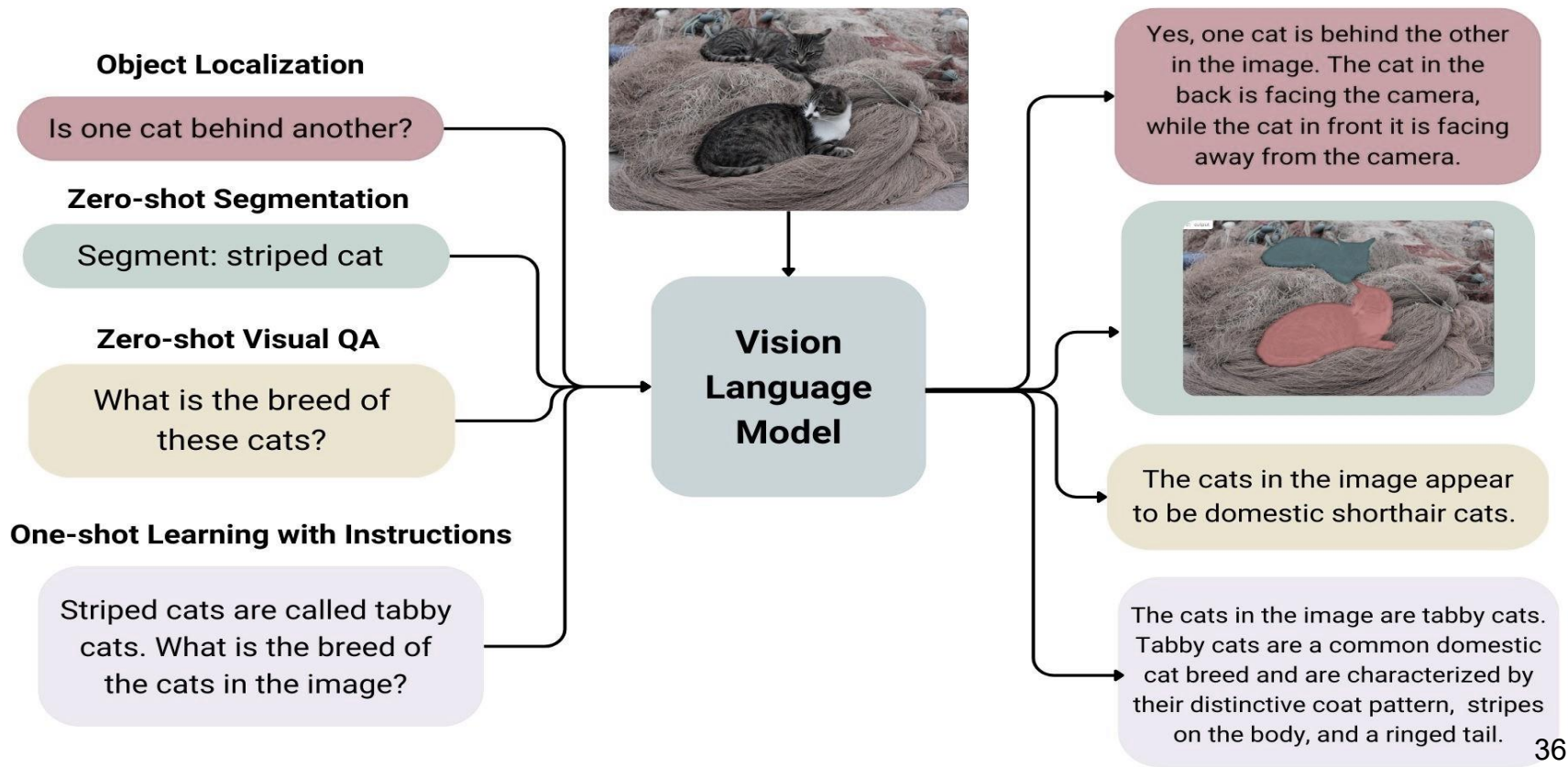
- Unimodal LLMs
 - Trained on a single modality of data, such as text
 - Lack visual context
- Multimodal LLMs:
 - Understand and generate content across multiple modalities,
 - such as text, images, audio, video, or other sensor data
- Vision-language models:
 - Multimodal models that can learn from images and text

Visual Language Models

- Main idea: Unify the image and text representation, and feed it to a textual decoder for generation
- Typically consists of two main components:
 - Vision encoder: extracts image features and encodes them into a format that can be understood by the language decoder.
Example: ResNet, ViT
 - Language decoder: takes these encoded visual features along with any textual input and generates descriptions, or captions, etc.
Example: GPT-2, GPT-3



VLMs



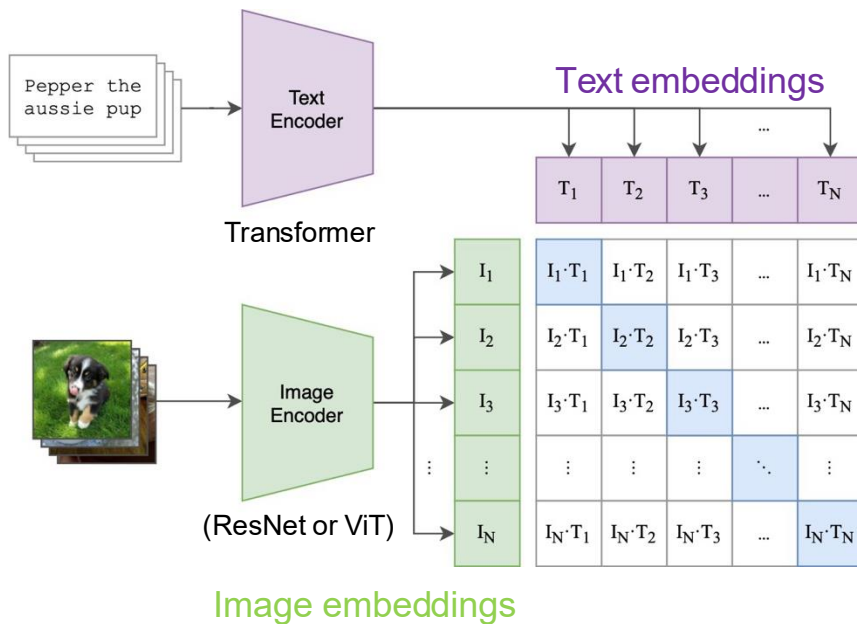
Contrastive Language-Image Pretraining (CLIP)

- CLIP differs from traditional vision-language models
 - It does not generate text descriptions or captions for images
 - Focuses on learning a joint representation space where images and text can be compared directly
- Enables various downstream tasks, such as
 - Zero-shot image classification: classify images into one of several classes, without any prior training or knowledge of the classes
 - Zero-shot image retrieval, text-based image generation

Contrastive Language-Image Pretraining (CLIP)

- Jointly trains a text encoder and an image encoder to predict the correct image—text pair

Contrastive Pre-training

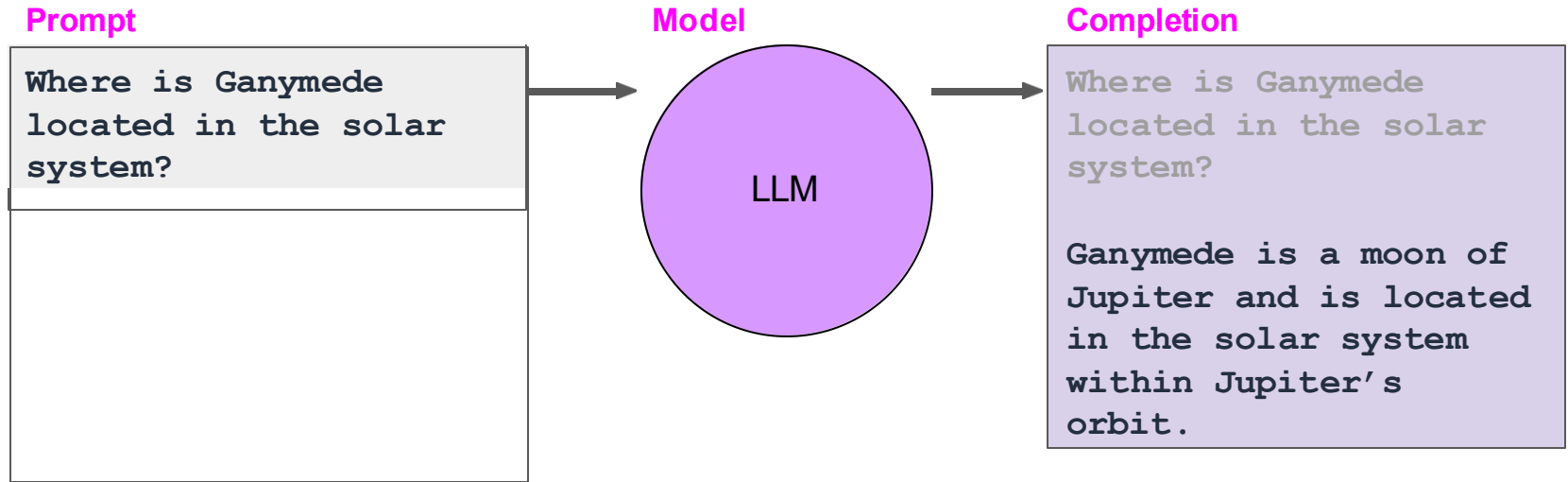


Contrastive learning framework

- Maximize the cosine similarities between correct image-text pairs
- Minimize the cosine similarities for dissimilar pairs (non-diagonal elements)

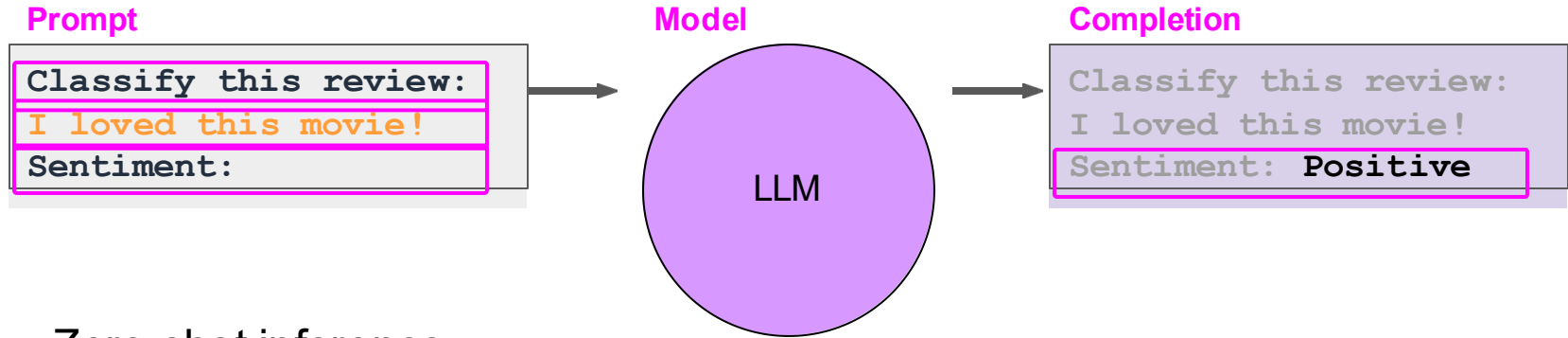
Prompting and prompt engineering

Prompting and prompt engineering



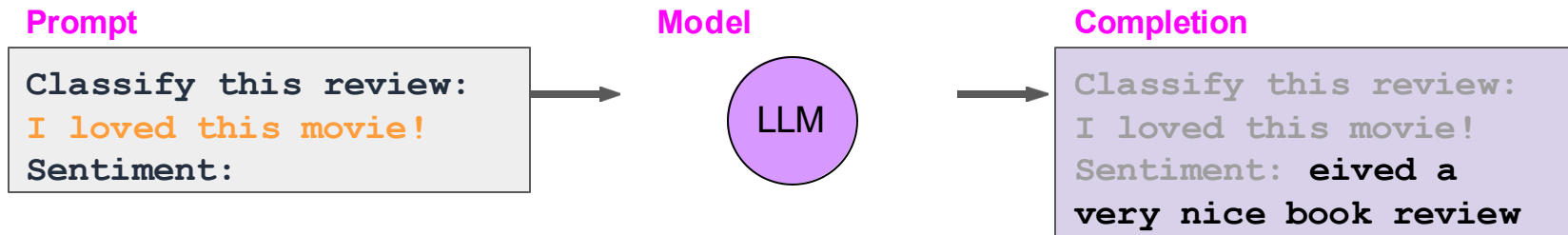
Context window: typically a few thousand words

In-context learning(ICL)- zero shot inference

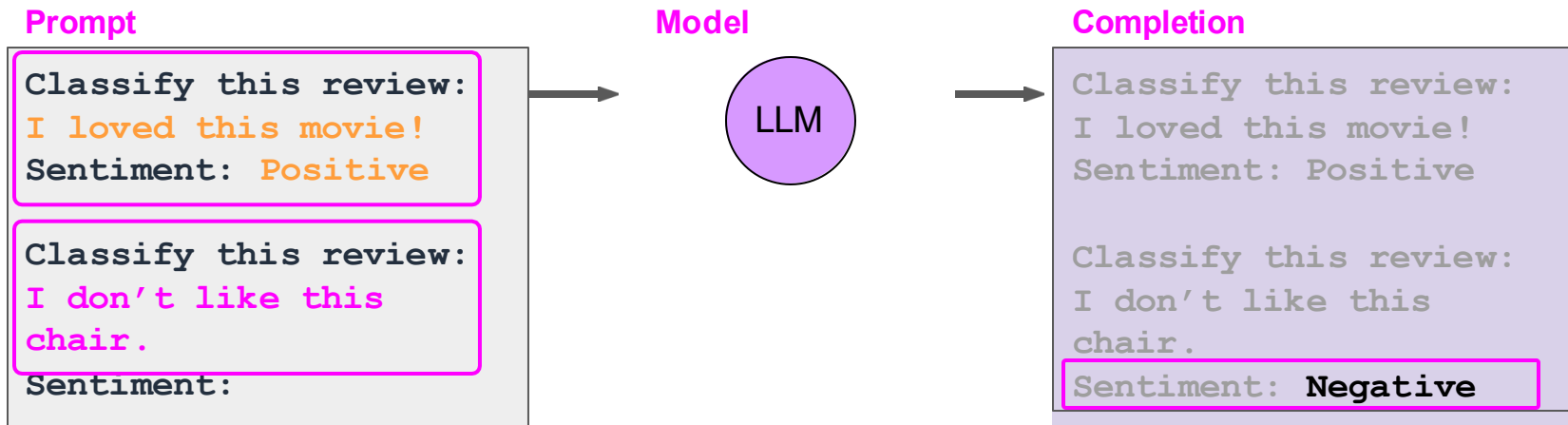


Zero-shot inference

In-context learning(ICL)- zero shot inference

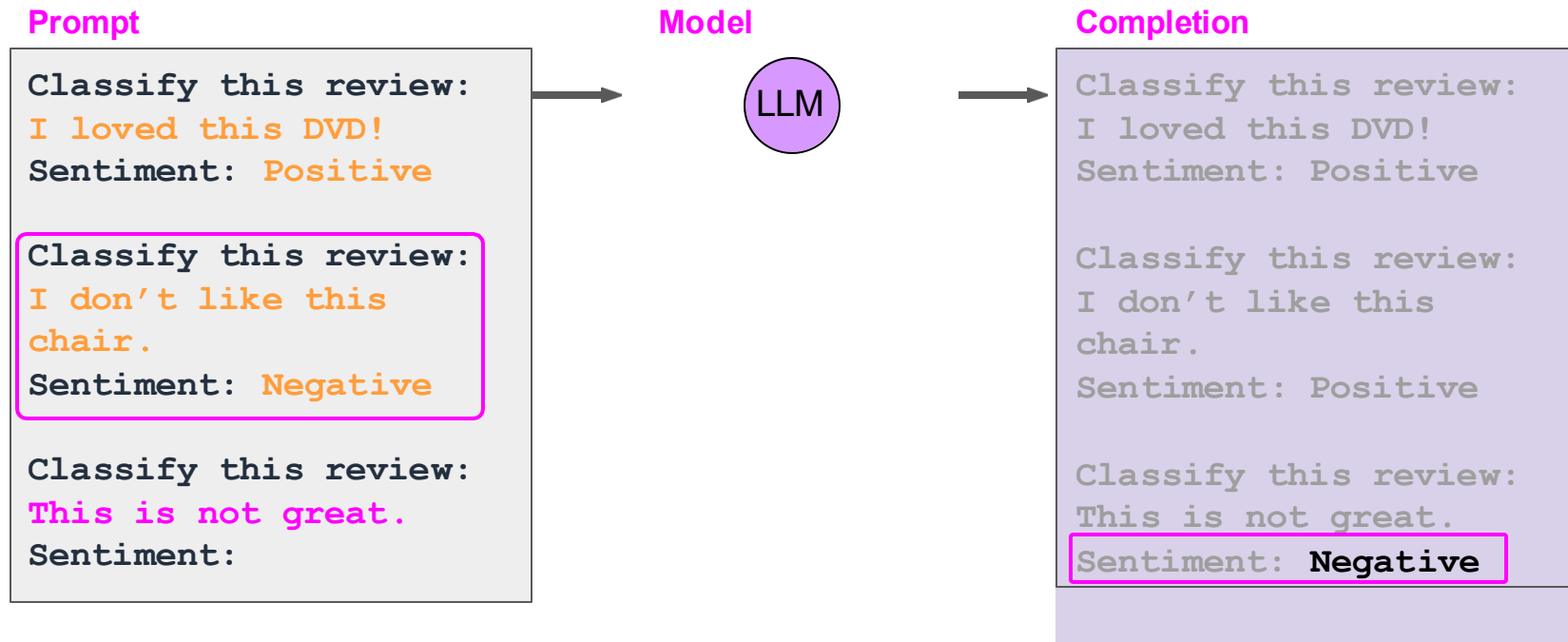


In-context learning (ICL) - one shot inference



One-shot inference

In-context learning (ICL)- few shot inference



Summary of in-context learning (ICL)

Prompt // Zero Shot

Classify this review:
I loved this movie!
Sentiment:

Prompt // One Shot

Classify this review:
I loved this movie!
Sentiment: Positive

Classify this review:
I don't like this
chair.
Sentiment:

Prompt // Few Shot >5 or 6 examples

Classify this review:
I loved this movie!
Sentiment: Positive

Classify this review:
I don't like this
chair.
Sentiment: Negative

Classify this review:
Who would use this
product?
Sentiment:

Context Window
(few thousand words)

Generative configuration parameters for inference

Generative configuration - inference parameters

Enter your prompt here...

Max new tokens 200

Sample top K 25

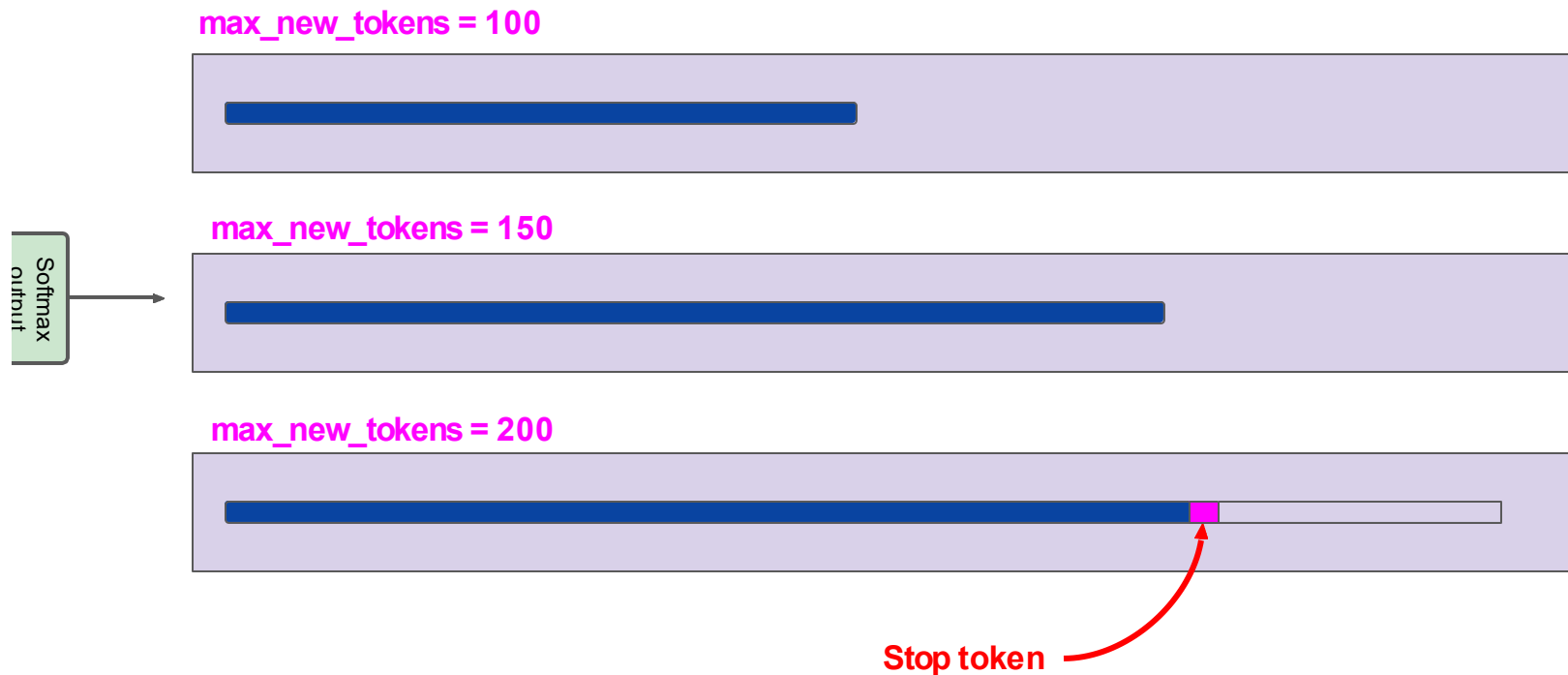
Sample top P 1

Temperature 0.8

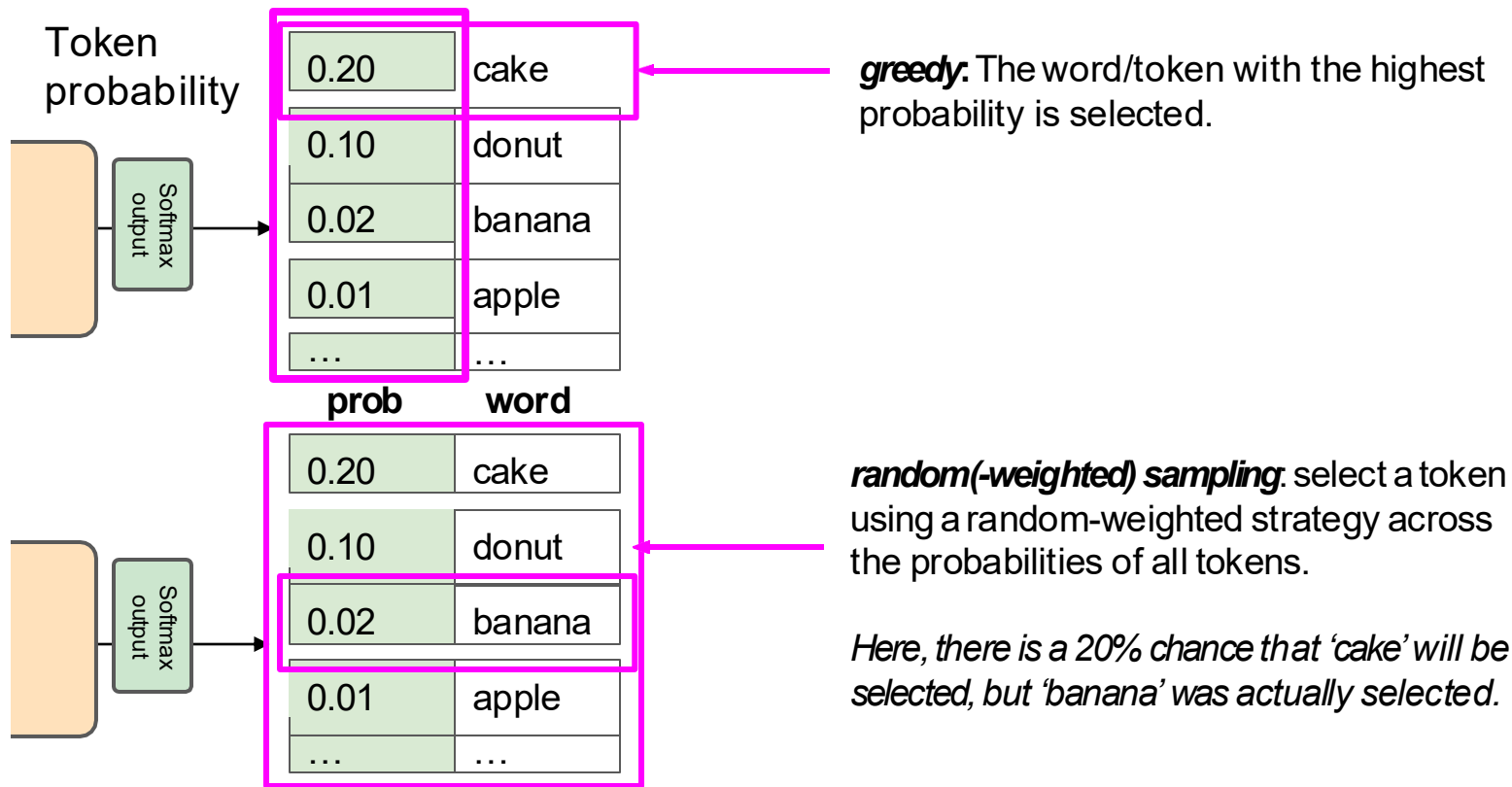
Submit

Inference configuration parameters

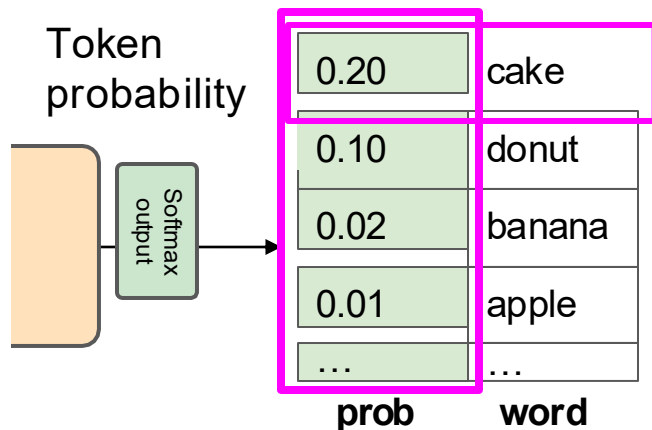
Generative config- max new tokens



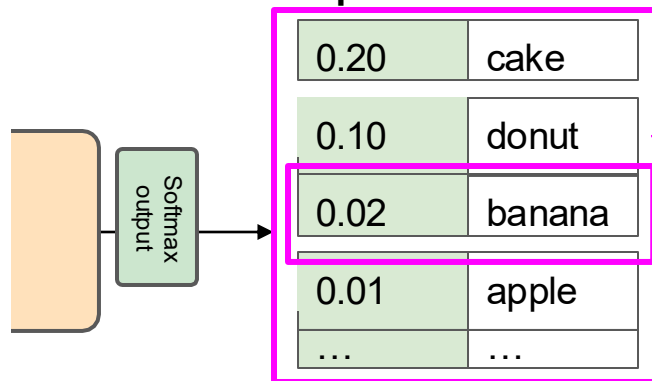
Generative config - greedy vs. random sampling



Generative config - greedy vs. random sampling



greedy. The word/token with the highest probability is selected.



random(-weighted) sampling: select a token using a random-weighted strategy across the probabilities of all tokens.

Here, there is a 20% chance that 'cake' will be selected, but 'banana' was actually selected.

Generative configuration - top-k and top-p

Enter your prompt here...

Max new tokens

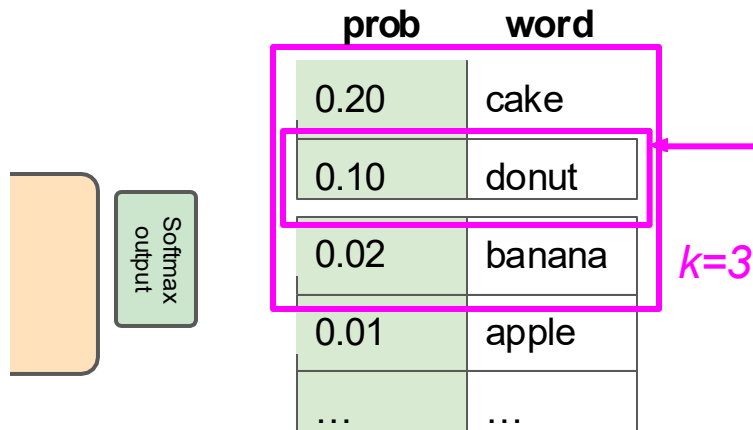
Sample top K

Sample top P

Temperature

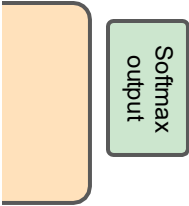
Top-k and top-p sampling

Generative config-top-k sampling



top-k: select an output from the top-k results after applying random-weighted strategy using the probabilities

Generative config-top-p sampling



prob	word
0.20	cake
0.10	donut
0.02	banana
0.01	apple
...	...

top-p: select an output using the random-weighted strategy with the top-ranked consecutive results by probability and with a cumulative probability $\leq p$.

$p = 0.30$

Generative config-temperature

Enter your prompt here...

Max new tokens

Sample top K

Sample top P

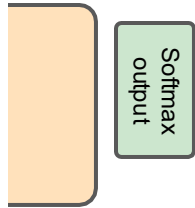
Temperature

Submit

Temperature

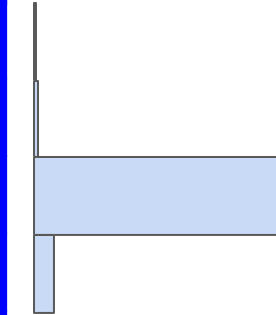
Generative config-temperature

Temperature
setting



Cooler temperature (e.g <1)

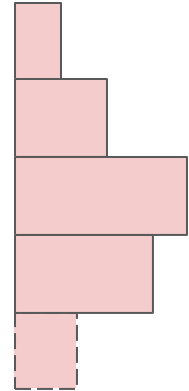
prob	word
0.001	apple
0.002	banana
0.400	cake
0.012	donut
...	...



Strongly peaked
probability
distribution

Higher temperature (>1)

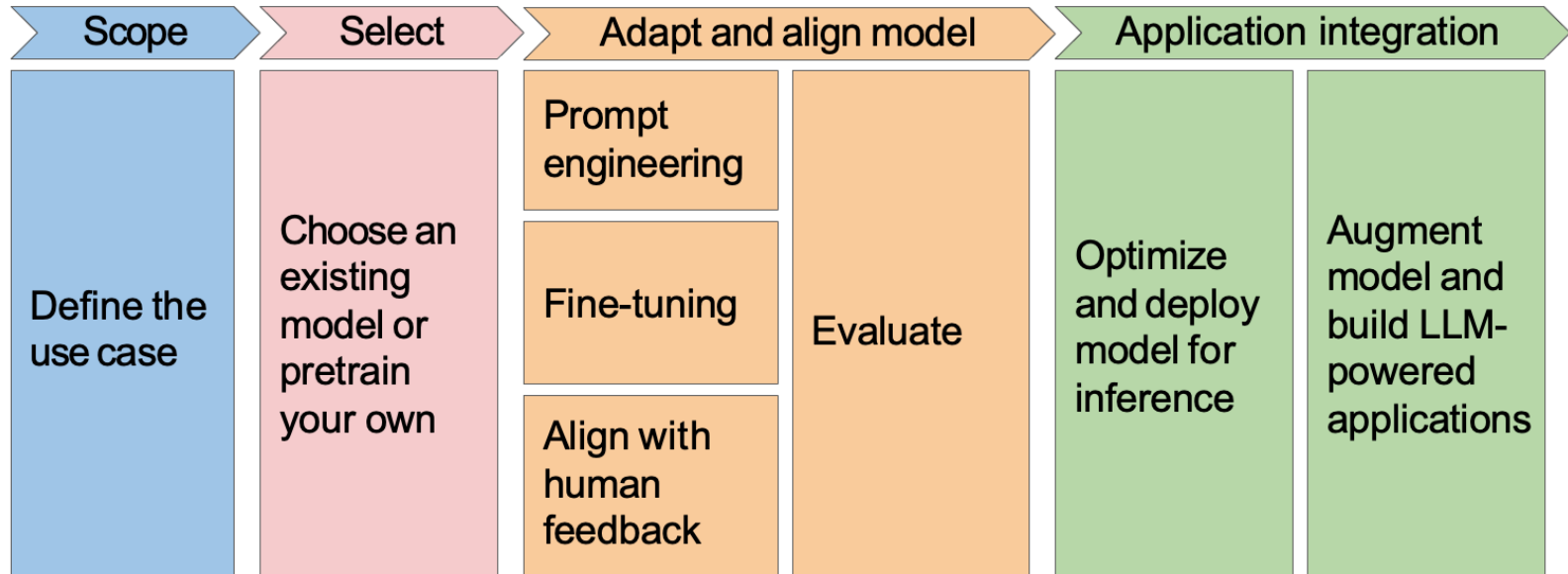
prob	word
0.040	apple
0.080	banana
0.150	cake
0.120	donut
...	...



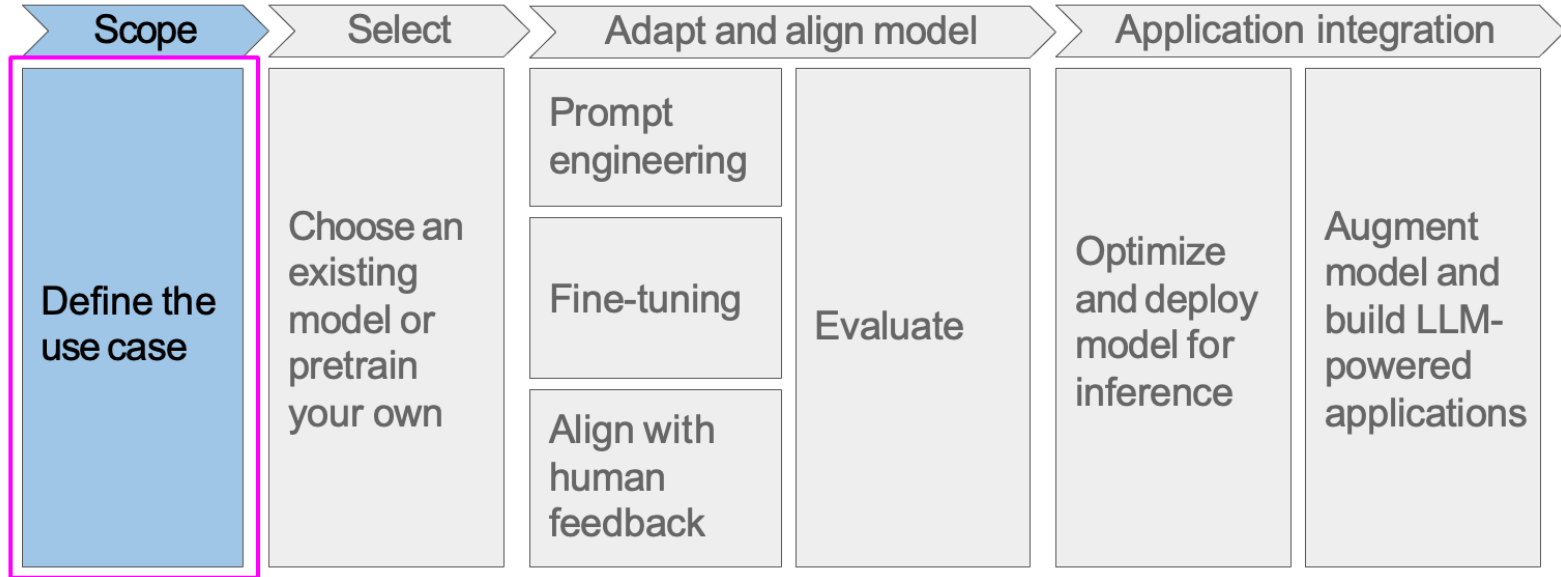
Broader, flatter
probability
distribution

Generative AI project lifecycle

Generative AI project lifecycle

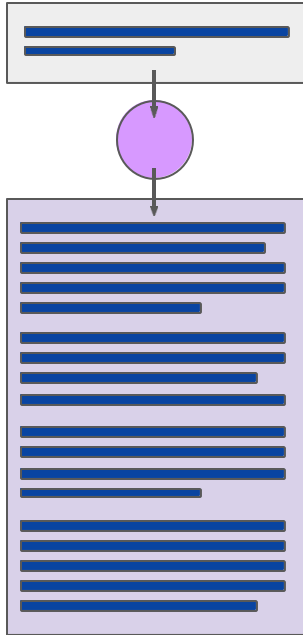


Generative AI project lifecycle

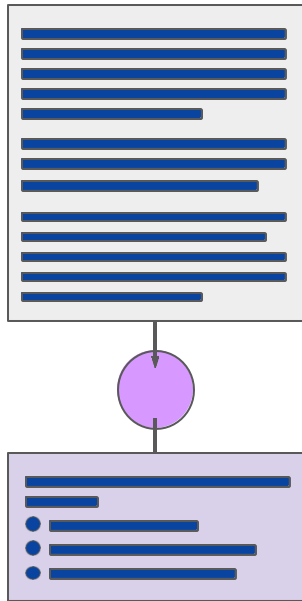


Good at many tasks / or a single task?

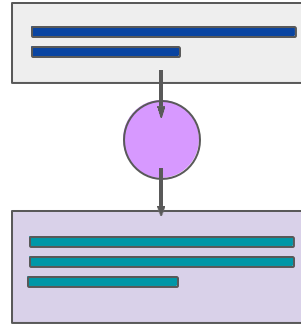
Essay Writing



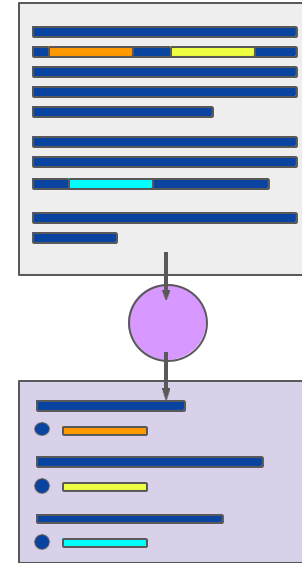
Summarization



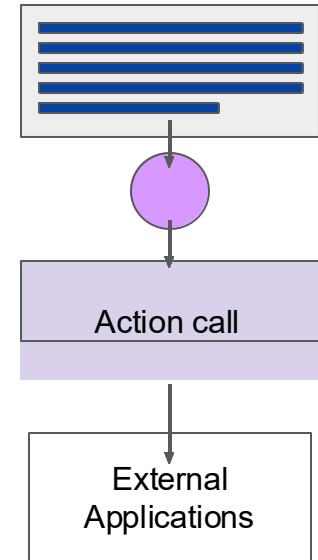
Translation



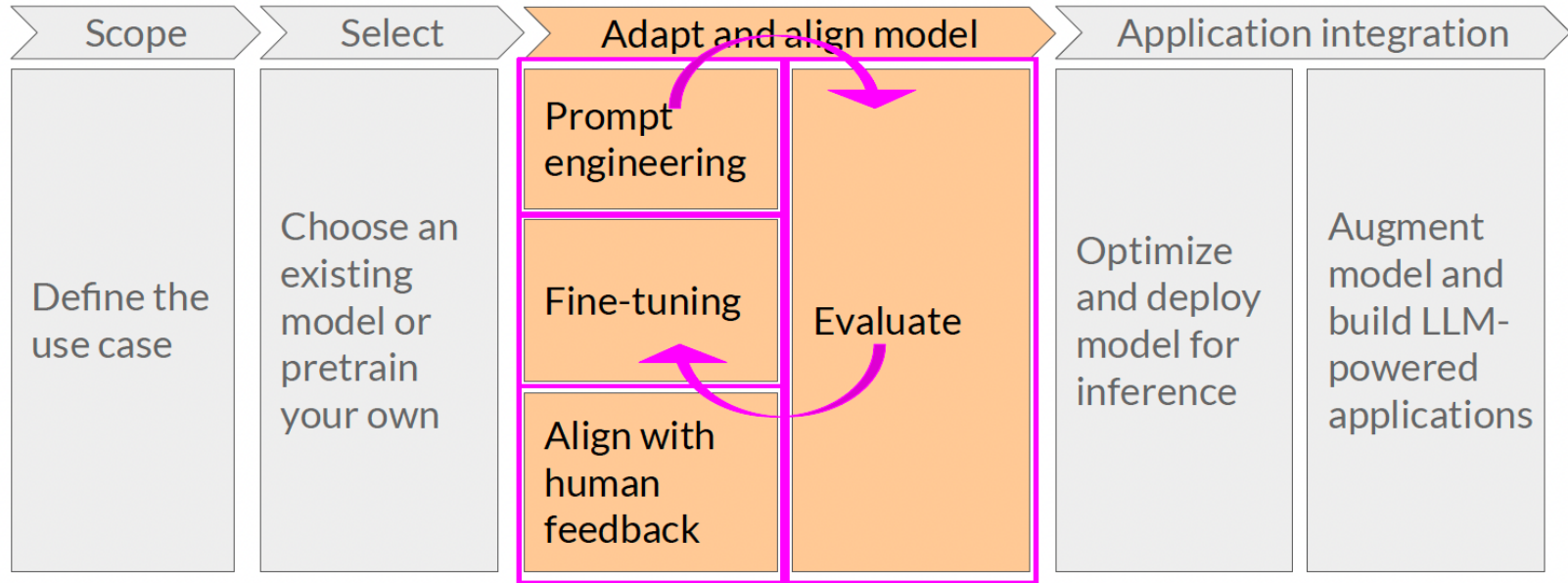
Information retrieval



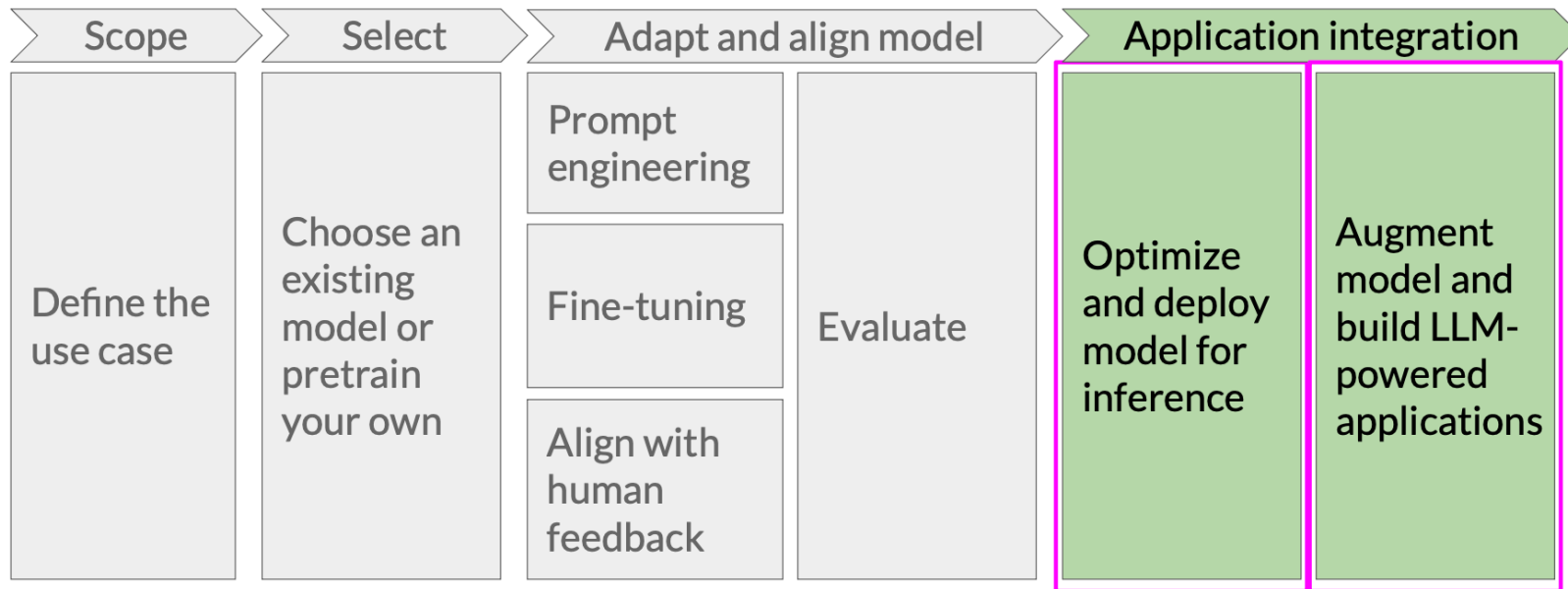
Invoke APIs and actions



Generative AI project lifecycle



Generative AI project lifecycle



Lecture References

- Hands-On Large Language Models: Jay Alammar & Maarten Grootendorst
- DeepLearning.AI – Generative AI for Everyone Course
- Introduction to Large Language Model (LLM) lecture: Ekta Vats
- A brief introduction to (large) language models lecture: Sachin Kumar